

---

# Spatial Mixture-of-Experts

---

Nikoli Dryden  
ETH Zürich  
ndryden@ethz.ch

Torsten Hoefer  
ETH Zürich  
htor@inf.ethz.ch

## Abstract

Many data have an underlying dependence on spatial location; it may be weather on the Earth, a simulation on a mesh, or a registered image. Yet this feature is rarely taken advantage of, and violates common assumptions made by many neural network layers, such as translation equivariance. Further, many works that do incorporate locality fail to capture fine-grained structure. To address this, we introduce the Spatial Mixture-of-Experts (SMOE) layer, a sparsely-gated layer that learns spatial structure in the input domain and routes experts at a fine-grained level to utilize it. We also develop new techniques to train SMOEs, including a self-supervised routing loss and damping expert errors. Finally, we show strong results for SMOEs on numerous tasks, and set new state-of-the-art results for medium-range weather prediction and post-processing ensemble weather forecasts.

## 1 Introduction

Many datasets exhibit an underlying, location-based structure, where the value at a point depends on where that point is. For example, weather predictions [78] depend on their location on Earth; many scientific simulations are parameterized on an underlying mesh [9]; data (e.g., faces) may be specifically aligned [91]; or it may approximately hold, as in natural images with centered objects. Further, tasks on such data are often dense regression tasks, such as predicting weather several days in the future. Numerous architectures have been successfully applied to such tasks. Convolutional neural networks (CNNs) [80] and transformers [74] show promise for medium-range weather prediction. Locally-connected networks (LCNs), which use independent, rather than shared, filters at each point, have been applied to weather post-processing [38], face recognition [46, 77, 91], and other tasks [18, 71], specifically to learn local features. Low-rank local connectivity (LRLCN) [30] relaxes the translation equivariance of CNNs while requiring fewer parameters than LCNs. Other approaches, such as CoordConv [65], add an additional inductive bias by providing explicit input coordinates.

However, for tasks on data with location-based structure, prior approaches suffer from various limitations. Convolution assumes that data is translation equivariant, which does not hold for such tasks [46, 91]. Approaches like LCNs require many parameters. **Many architectures have been designed for classification tasks and fail to perform well on regression because they operate at too coarse granularity and are unable to capture key details.** This limits their applicability to important tasks, such as medium-range weather prediction or climate simulations. Indeed, on a simple heat diffusion task with location-dependent diffusivities (see §3.1), many approaches do not learn the location dependence at all and instead converge to an “average” diffusivity.

To address this, we introduce a novel neural network layer, the Spatial Mixture-of-Experts (SMOE) layer (§2). An SMOE uses a learned gating function to **sparsely select and route from a shared set of experts to each spatial location** (e.g., **pixel**) in an input sample. This enables experts to specialize to the unique characteristics of different “regions” within the data and easy interpretability by analyzing the gate. SMOEs require the assumption that all samples in the dataset have a similar underlying spatial structure, but this often holds (at least approximately) for many datasets, such as weather, where each example is on the same latitude/longitude grid. For the SMOE gating function, we introduce *tensor routing*, a simple and cheap routing function that effectively learns this spatial

36th Conference on Neural Information Processing Systems (NeurIPS 2022).

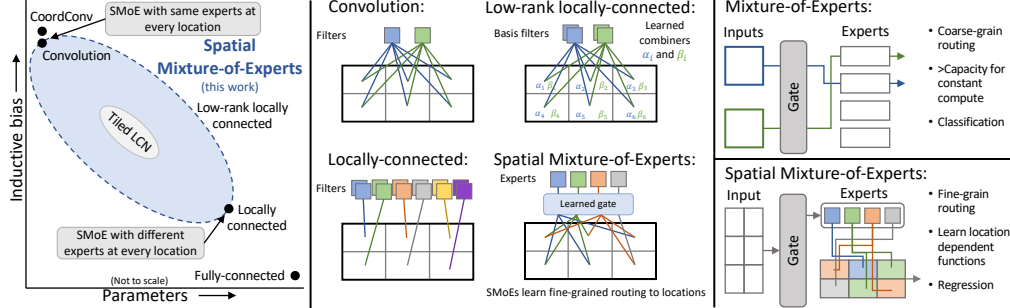


Figure 1: Comparison of SMOEs to other networks. **Left:** Qualitative comparison of parameter counts and inductive biases. **Center:** Pattern of filter application among models. **Right:** Routing in MoEs and SMOEs.

structure (§2.1). We also find that end-to-end training approaches, as in standard mixture-of-experts (MoEs) [82, 86], do not capture location-dependence well. Instead, we introduce a separate *routing classification loss* (§2.2) to train the gate, a self-supervised loss which provides a better learning signal to the gate about incorrect routings. We also introduce *expert error damping* (§2.3), which limits expert updates from misroutings, and further improves performance. Both methods rely on extracting information from the error signal (gradient of the layer output) which would otherwise be uninformative. Figure 1 provides an overview of SMOEs and a qualitative comparison to important related work. With these methods, SMOEs are able to learn fine-grain, location-dependent structure which other architectures miss, and consequentially deliver much better performance.

SMoEs are related to and inspired by prior work in MoEs (e.g., [82, 86, 102]), but there are key differences. Existing MoEs use coarse-grained routing at the sample [99], token [86], or patch [82] level, and experts produce coarse output (e.g., entire samples or channels) while SMOEs route at a fine-grained, per-pixel level. Fine-grained routing is important for enabling experts to specialize to fine-scale information, which may be crucial [74, 84]. Such MoEs also typically aim to increase model capacity while keeping compute costs constant, whereas the goal of SMOEs is to capture spatial dependencies. Other work has aimed to incorporate spatial dependence, such as LRLCNs [30], which learn content- and location-dependent weights to combine a set of basis filters. However, we found that these prior methods failed to capture the fine-grained features necessary for good performance on dense regression tasks (§3.1).

We conduct experiments on several benchmark datasets with SMOEs and conduct extensive ablation studies of SMOE design decisions (§3). Using a simple heat diffusion dataset, we showcase the limitations of other models and the power of SMOEs in a controlled situation. We then apply SMOEs to medium-range weather prediction and outperform the state-of-the-art [53, 80] on WeatherBench [78]; and set a new state-of-the-art for post-processing ensemble weather forecasts on the ENS-10 dataset [8]. Finally, we show that SMOEs can also be applied to image classification tasks, where we match or outperform LRLCNs while using fewer parameters.

Our code is available at <https://github.com/spcl/smoel>.

## 1.1 Related Work

**Mixture-of-Experts and conditional computation.** While MoEs have long been used [15, 49, 52], since the advent of deep learning, there has been much work in applying MoEs, conditional computation, and dynamic routing to DNNs [1, 4, 5, 7, 10–12, 19–21, 23, 25, 27, 29, 33, 34, 39, 51, 59, 60, 70, 75, 76, 82, 83, 86, 99, 102, 107]. Often, the goal is to increase model capacity without a corresponding increase in compute costs and models use coarse-grained routing. Notably, MoEs route at the sample (e.g., [25, 29, 70, 83]), token (e.g., [33, 59, 86]), or patch (e.g., [82]) level; are typically trained with extensive auxiliary losses; and often target classification tasks. This coarse routing, in particular, means there is limited opportunity for experts to specialize. MoEs specifically for vision tasks also typically operate at a sample-level (e.g., [1, 4, 39, 99, 102]) and use experts to specialize filter or channel choices. In contrast, our SMOEs induce an inductive bias via fine-grained, location-dependent routing for problems with such underlying structure; typically do not need auxiliary losses beyond the routing classification loss; and work well for dense regression tasks.

**Local spatial structure.** Locally connected networks (i.e., convolution filters without weight sharing) were historically successful in vision tasks such as facial recognition [18, 46, 77, 91]. Further

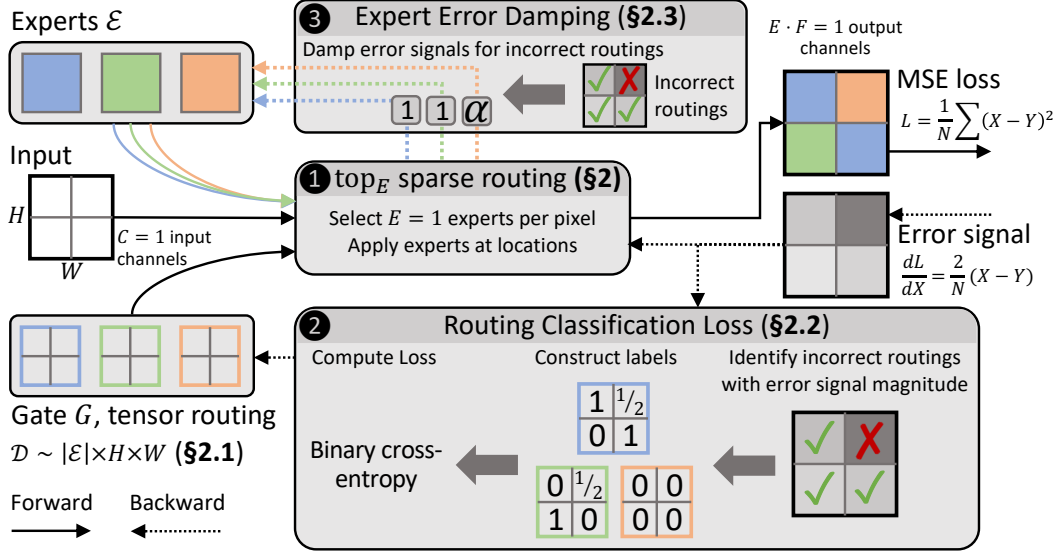


Figure 2: Overview of SMOE architecture and training using a mean-square error loss and an example input. ① Each location in an input is routed to  $E$  experts from a set  $\mathcal{E}$  based on a gate  $G$  (§2 and 2.1). ② The gate is trained using a routing classification loss, which identifies incorrect routings based on the error signal (§2.2). ③ The error signal to experts is damped for locations they were incorrectly routed to (§2.3).

work incorporated periodic or tiled weight sharing [37, 71, 105]. However, more recent work often exclusively uses convolution, which has been found to perform better [72] while requiring significantly fewer parameters. CoordConv [65] explicitly provides pixel coordinates as input. Because of their structure, a single convolution or CoordConv layer cannot learn location-dependent sparse activations as SMOEs do, and multi-layer networks give limited improvements in practice. Low-rank locally connected networks [30] learn to combine a small number of basis filters in a location- and input-dependent manner. These combiners lack sparsity and apply all basis filters at each point, and are softmax-normalized, which can result in a few filters dominating, limiting diversity. Separate work has used attention to provide position-dependent scaling or integrate context [35, 50, 64, 93, 95, 100]. This has culminated in vision transformers [26] and variants [28, 62, 67, 96, 97], which use specialized attention mechanisms to integrate spatial information. Other work uses Markov [61, 66] or conditional random fields [16, 43, 106], or graph-based methods [17, 98, 104], to learn long-range dependencies. Additional work has studied incorporating equivariance properties into neural networks (e.g., [13]).

**Sparsity.** Our use of sparse routing resembles work on sparsity in DNNs in general [45]. Many approaches learn a fine-grained mask to identify which weights to keep or prune (e.g., [63, 90]), while other approaches sparsify activations (e.g., [3, 47, 56, 68, 69]). These works use sparsity to improve runtime performance, typically at inference time.

**Weather prediction.** Medium-range weather prediction, or forecasting roughly three to seven days in advance [6], is of broad societal import [57]. There has been much interest in applying deep learning to this task [85], and WeatherBench [78] serves as a community benchmark. Deep learning has also been successfully applied to the related tasks of ensemble post-processing [38, 79] and now-casting (forecasting a few hours in advance) [2, 31, 81, 87–89]. Standard CNNs are currently state-of-the-art on WeatherBench [80] (although graph neural networks show promise on similar data [53]) and are competitive on post-processing tasks [8]. SMOEs can take advantage of the extensive fine-grained, location dependent structure in weather data for improved performance.

## 2 Spatial Mixture of Experts

We now introduce the Spatial Mixture-of-Experts (SMOE) layer. An SMOE uses a learned gating function (§2.1) to select specific experts from a shared set to apply at each point (e.g., pixel) in an input. The gate learns the underlying spatial structure of the data, and routes specific experts to different “regions”, allowing them to specialize. An SMOE is predicated on the assumption that the spatial structure is similar across all samples in a dataset. We also assume data are on a Cartesian mesh (i.e., grid), although this could be generalized. To train SMOEs, we introduce a self-supervised

*routing classification loss* (§2.2) and *expert error damping* (§2.3), which we found essential for achieving the best performance. Figure 2 provides an overview of SMOEs and their training.

We first define the SMOE layer in full generality, then discuss the particular implementation we use. (See §3.1 for ablations.) Let  $x \in \mathbb{R}^{C \times H \times W}$  be an input sample (we assume 2D data for simplicity). The SMOE layer consists of a set  $\mathcal{E}$  of experts, of which  $E \leq |\mathcal{E}|$  will be selected at each point, and a gating function  $G : \mathbb{R}^{C \times H \times W} \rightarrow \mathbb{R}^{|\mathcal{E}| \times H \times W}$ . The gating function has exactly  $E$  nonzeros at each spatial point ( $H \times W$ ), which correspond to the selected experts at that point. Each expert  $e \in \mathcal{E}$  is applied at the points in  $x$  where it has been selected, and may include additional context from  $x$  (e.g., surrounding points if  $e$  is convolution), and produces an output of size  $F$ . The outputs of each expert at a point are then concatenated to form the output channel dimension (of size  $E \cdot F$ ) of the SMOE. More precisely, let  $\text{gather}_I(\cdot)$  select only the input entries where  $I$  is nonzero. Then an SMOE is:

$$y = \text{gather}_{G(x)}(G(x) \cdot [e_1(x); \dots; e_{|\mathcal{E}|}(x)]),$$

where  $\cdot$  is element-wise multiplication,  $[e; \dots]$  stacks tensors, and  $y \in \mathbb{R}^{EF \times H \times W}$ . When  $E \ll |\mathcal{E}|$ , the gating function induces significant sparsity; this allows an SMOE to compute experts only at the points where they are to be applied, avoiding most computation.

This formulation yields a *weighted* SMOE, where the expert outputs are scaled by the gating function. An alternative, which is slightly more efficient and can be more readily interpretable, is an *unweighted* SMOE, in which case experts are not scaled, and the gather only uses  $G(x)$  to select experts.

In this work, we focus on a simple, yet powerful, SMOE layer using convolution filters as experts:

$$y_i = G(x)_i \cdot \sum_{c \in [C]} w_{i,c} \star x_c,$$

where  $\star$  is cross-correlation,  $i \in [|\mathcal{E}|]$ , and we have elided the gather for simplicity.

## 2.1 Gating Functions

The gating function  $G$  in an SMOE is critical for learning the underlying location-dependence of the data. While  $G$  can be any function, a good gate should be relatively cheap. We follow prior work on MoEs (e.g., [82, 86]) and use *top- $E$  routing* to select experts and sparsify the gate:  $G(x) = \text{top}_E(g(x))$ , where  $g(x) \in \mathbb{R}^{|\mathcal{E}| \times H \times W}$  is a learnable gating layer and  $\text{top}_E$  selects the  $E$  largest entries along the first (expert) dimension and sets the remaining entries to 0. However, we found that normalizing the gating function using softmax, as is typically done, was not necessary.

There are many options for the gating layer. Many MoE works have used MLPs [82, 86], but we found this did not perform well. Convolution or CoordConv [65] also did not perform well (§3.1). Instead, we find that a simple *tensor routing* gating layer worked best. With this,  $g(x) = \mathcal{D} \in \mathbb{R}^{|\mathcal{E}| \times H \times W}$ , where  $\mathcal{D}$  is a learnable tensor that directly encodes the underlying location dependence and routing structure *without depending on the input*. Further,  $\mathcal{D}$  uses one parameter per expert per location and requires no computation beyond optimizer updates, making it an efficient choice.

We initialize  $\mathcal{D}$  using a uniform distribution over  $[-3|\mathcal{E}|/EF, 3|\mathcal{E}|/EF]$ , which corresponds to a Kaiming uniform initialization with fan-in  $EF/|\mathcal{E}|$  [41]. However, in many practical cases, some data about locations may be known (e.g., whether it is land or sea when doing weather prediction). In such cases,  $\mathcal{D}$  can be initialized based on this data to assign groups of experts in advance. This allows the gating function to benefit from prior knowledge while still being able to adjust the routing. Additionally, a network may contain many SMOE layers, in which case we can share  $\mathcal{D}$  between layers that have the same spatial dimensions and number of experts, which reduces the overhead of the gate.

Finally, many MoE formulations use a number of auxiliary losses to ensure gates learn good routing policies and avoid mode collapse (e.g., [10, 82, 86]). When training with the routing classification loss we propose, *we did not find additional auxiliary losses to be necessary*.

## 2.2 Training and the Routing Classification Loss

Training MoEs is typically done end-to-end, with the *experts and gate learning simultaneously, and sparse gradients based on the routing*. We found this did not lead to good performance with SMOEs and tensor routing, particularly on regression tasks, as the gate did not learn well: it rarely changed its routing decisions from initialization. We hypothesize that this is due to a “mismatch” in

the gradients on regression tasks, where they are informative for experts but not the gate, because regression aims to make a continuous prediction over both positive and negative values, whereas selecting an expert requires a threshold (see §B). To address this, we train the gate with a separate, self-supervised loss function, the *routing classification (RC)* loss. The key idea is to consider routing as a dense, multi-label classification task: selecting the “correct” experts at each point (cf. semantic segmentation). The RC loss does exactly this, and trains the gate by constructing appropriate labels. This also helps avoid mode collapse, where only a small number of experts are selected (see §3.2).

In order to construct these labels, we need to determine whether the gate selected the correct set of experts at each point. However, such information is not directly available. Instead, we use the error signal into the SMOE layer (i.e., the gradient of the layer output w.r.t. the loss) as a proxy, and say that the routing was incorrect when the error signal has a large magnitude for the expert at that point, as this will imply a correspondingly large gradient update. Further, in the case of a mean-square error loss  $L$  (commonly used for regression), the error signal can directly encode the prediction error. Let  $X$  be the predictions,  $Y$  the true values, and  $N$  the number of elements in  $X$ . Then the error signal is:

$$\frac{dL}{dX} = \frac{d}{dX} \frac{1}{N} \sum (X - Y)^2 = \frac{2}{N} (X - Y).$$

Hence, the error signal is simply the (scaled, signed) error of the predictions. While this exact relation ceases to hold as backpropagation proceeds, the intuition behind the error signal magnitude remains.

We now define the routing classification loss. Given the error signal  $\varepsilon$  into an SMOE layer, and an error quantile  $q$  (a hyperparameter), we say that selecting an expert at a point was incorrect if  $\varepsilon$  at that expert and point is greater than the  $q$ th quantile of  $\varepsilon$ , and correct otherwise. We use quantiles as they are independent of  $\varepsilon$ ’s scale, which may be hard to determine and change during training. We then construct the labels for each point as follows: Unselected experts start with label 0. A correctly selected expert has label 1. Finally, if an expert was incorrectly selected, its label is 0 and we add  $1/(|\mathcal{E}| - E)$  to the label value of each unselected expert (note  $|\mathcal{E}| - E$  is the number of unselected experts). This corresponds to a uniform prior that the correct expert could be any unselected expert. With these labels, the RC loss for a gate is then the binary cross-entropy loss of the gate output.

### 2.3 Expert Error Damping

While the RC loss enables an SMOE gate to be trained directly based on whether it routed correctly, experts that were incorrectly routed to still perform gradient updates based on this routing. This results in experts updating to improve their performance for locations they may not be applied at in future training iterations after routing changes. To mitigate this, we propose *expert error damping*, where the portion of an error signal that corresponds to incorrect routings is damped to limit incorrect updates. We find that this can improve SMOE performance and reduce convergence time.

Expert error damping is similar to the RC loss, and we classify incorrect routings in the same way. We then scale the error signal into the experts by a constant factor at each point where the routing was incorrect. This will limit the magnitude of the update made in response to the misrouting.

### 2.4 Practical Implementation

We now discuss the implementation of an SMOE layer, primarily focusing on the simple SMOE with convolution filters we use. In an ideal implementation, the flop cost of an SMOE is the cost of the gate plus the cost of applying the selected experts. When using a tensor routing gate, there are no flops in the gate. The flops from applying the selected convolutional experts is equivalent to a standard convolutional layer with a number of filters equal to the number of selected experts. Hence, SMOEs are quite efficient flop-wise. However, recent work has shown that data movement is also critical to performance [48]. Because of this, we do not expect even well-optimized implementations to match the runtime of a convolution layer due to the sparse routing, limited locality in accessing expert filters, and other operations, although work on hardware-accelerated sparsity [22] should offer benefits. Despite this, during inference additional optimizations may be available because the tensor routing gate does not depend on the input and computations could be reordered to maximize locality.

Unfortunately, efficiently implementing the irregular access patterns in an SMOE is challenging in standard Python frameworks, and likely requires custom compute kernels. Instead we opt for a naïve implementation in PyTorch [73] where we apply all experts at all points and then use gather and scatter operations to implement sparse routing. Experts are concatenated in sorted routing score



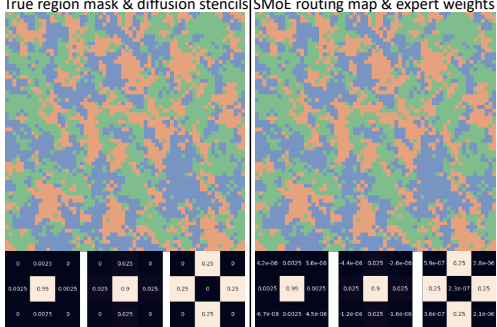
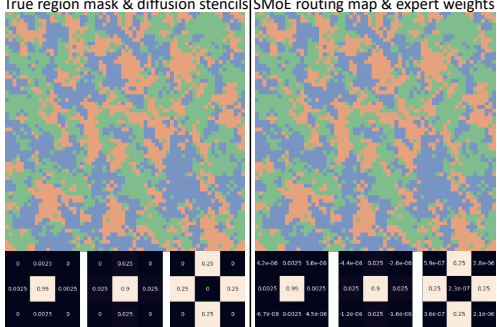
| Model          | #Params  | Epochs | % within 1%    | True region mask & diffusion stencils                                              | SMoE routing map & expert weights                                                  |
|----------------|----------|--------|----------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Convolution    | 146      | 102    | 91.3 $\pm$ 0.2 |  |  |
| CoordConv [65] | 56       | 50     | 91.1 $\pm$ 0.2 |                                                                                    |                                                                                    |
| CondConv [102] | 200      | 120    | 91.2 $\pm$ 0.3 |                                                                                    |                                                                                    |
| LRLCN [30]     | 516      | 27     | 91.8 $\pm$ 0.5 |                                                                                    |                                                                                    |
| LRLCN-ND [30]  | 12315    | 35     | 91.5 $\pm$ 0.3 |                                                                                    |                                                                                    |
| LCN            | 36764    | 110    | 100.0 $\pm$ 0  |                                                                                    |                                                                                    |
| FC             | 16.7M    | 250    | 14.2 $\pm$ 1.2 |                                                                                    |                                                                                    |
| ViT [26]       | 200k     | 67     | 93.5 $\pm$ 0.1 |                                                                                    |                                                                                    |
| V-MoE [82]     | 470k     | 74     | 93.8 $\pm$ 0.3 |                                                                                    |                                                                                    |
| SMoE           | 27+12288 | 8      | 100.0 $\pm$ 0  |                                                                                    |                                                                                    |

Figure 3: Heat diffusion results. **Left:** SMoE and baseline performance. **Center:** Dataset region map and diffusion stencils. **Right:** SMoE routing map and experts. SMoEs learn correct stencils and location-dependence.

order (i.e., as given by  $G(x)$ ); while this can result in channel orders changing during training, we find it has little impact (see §3.2). We find this implementation sufficient even for large-scale tasks.

### 3 Experiments

We now present experimental results using SMoEs and a variety of baseline models, as well as ablations of different SMoE components. First, we describe a simple location-dependent heat diffusion dataset, which we use to study the ability of different architectures to learn location dependence in a controlled environment (§3.1). We then present results on the WeatherBench [78] medium-range weather forecasting challenge (§3.2) and the ENS-10 [8] dataset for post-processing ensemble weather forecasts (§3.3), where SMoEs set new state-of-the-art performance results. Lastly, we show results on several image classification tasks, illustrating the breadth of SMoE applicability even in situations without strict location-dependence (§3.4).

All results were run using PyTorch [73] version 1.11 on a large cluster with 16 GB V100 GPUs. We summarize training details throughout this section, and provide full details in §A. We use existing PyTorch implementations when available, and implemented methods ourselves otherwise. Unless noted, all SMoE results used **3×3 convolution kernel experts**, unweighted tensor routing gates, RC loss, expert error damping, error quantile  $q = 0.7$ , and damping factor **0.1**. We report the mean and standard deviation (mean $\pm$ std) over ten runs for all experiments, except for WeatherBench and ImageNet, which used three runs. In total, we used about **30k GPU hours** for experiments.

#### 3.1 Location-Dependent Heat Diffusion

To study location-dependence in a controlled manner, we generated a heat diffusion dataset with location-dependent diffusivities. The task is to predict what the heat will be at the next timestep, given the current heat at each point. Because we know the exact diffusivities and their distribution in the data, it is easy to identify how well a model has learned this task. We first describe the dataset and its generation in detail, then discuss results and ablation studies on SMoE components.

**Dataset.** The dataset consists of a region map, where each location is assigned a type, and each type corresponds to a different heat diffusivity. The region map and diffusivity assignment are then fixed for the dataset. We generate the region map randomly using a simple preferential attachment method. This defines the location-dependence that we wish to learn. To generate samples, we first randomly distribute drops of heat across the domain, then apply five-point diffusion stencils at each point, using the diffusivity of the region type for each point. For simplicity, we use zero boundary conditions. This process is iterated to generate many timesteps from the given starting state. The dataset then consists of the generated timesteps for many different starting states.

If a model is able to learn the diffusion stencils and the location-diffusivity correspondence, it can exactly predict the next timestep. Further, the diffusivity stencils are also simple, and exactly correspond to a 3×3 convolution kernel with no nonlinearity.

The particular dataset we use consists of 100,000 64×64 samples, with 1,000 initial states evolved for 100 timesteps each. Adding more samples did not significantly change results. There are three region types, with diffusivities 0.25, 0.025, and 0.0025. Figure 3 (center) shows the region map and

Table 1: Effect of RC loss and expert error damping on the heat diffusion dataset.

| RC loss | Damping | Epochs | % within 1%          |
|---------|---------|--------|----------------------|
| ✗       | ✗       | 14     | 91.5 $\pm$ 0.4       |
| ✗       | ✓       | 16     | 91.6 $\pm$ 0.2       |
| ✓       | ✗       | 15     | 96.7 $\pm$ 0.4       |
| ✓       | ✓       | 8      | <b>100.0</b> $\pm$ 0 |

Table 2: Effect of different SMOE gate functions on the heat diffusion dataset.

| Gate               | #Params | % within 1%          |
|--------------------|---------|----------------------|
| Fully-connected    | 50M     | 85.2 $\pm$ 5.7       |
| 3×3 convolution    | 27      | 91.3 $\pm$ 0.3       |
| 3×3 CoordConv [65] | 81      | 91.5 $\pm$ 0.4       |
| 3×3 LCN            | 110592  | 96.3 $\pm$ 1.2       |
| 3×3 CoordConv×3    | 255     | 91.6 $\pm$ 0.2       |
| Tensor routing     | 12288   | <b>100.0</b> $\pm$ 0 |

diffusion stencils. We report results using “% within 1%”, the percentage of locations in a sample that are within 1% relative error of the true value, as this is more interpretable than a mean-square error.

**Results.** Figure 3 (left) shows results on the heat diffusion dataset for SMOEs and a number of baselines: CNNs, LCNs, fully-connected (FC) layers, CoordConv [65], CondConv [102], LRLCNs [30], vision transformers (ViT) [26], and vision MoEs (V-MoE) [82]. For the LCN and FC layers, we use only a single layer as additional ones showed no benefit. Convolution, CoordConv, CondConv, and LRLCN are the best network found among a set with up to three layers, 3×3 kernels, 12 filters per layer, batchnorm, and ReLU activations. ViT and V-MoE use one transformer block with a patch size of 4×4, an embedding dimension of 128, and four heads. LRLCNs use three basis filters and an input-dependent combiner. We also tried unshared combining weights with no input dependence (LRLCN-ND). V-MoEs select one expert from a set of three. Our SMOEs use a single layer with three experts and select  $E = 1$  expert per point. All models were trained with batch size 32, Adam [54] with a learning rate of 0.001 (decayed by 10× after no validation improvement for 15 epochs), and early stopping after no improvement for 30 epochs. Additional hyperparameter tuning did not significantly improve results. We report SMOE parameters as expert+gate parameters.

SMOEs achieve perfect performance on this dataset. Further, by examining the learned routing and experts (Fig. 3, right), we can see that it has indeed correctly learned the diffusion stencils and location-dependence. LCNs also achieve this, but require 3× more parameters, and require 110 epochs to converge (versus 8 for SMOEs). Fully-connected layers failed to learn the data well, likely due to the challenge of optimizing so many parameters. Other methods all converge to between 91 and 94% within 1%. Examining their predictions and weights, we observe that they do not appear to have learned the location-dependence of the diffusivities, and instead converged to predicting with an “average” diffusivity across the domain. We also tried larger (deeper and/or wider) convolutional networks, but performance did not improve. MoE methods (CondConv and V-MoE) also fail in this manner, as their coarse-grained experts are unable to specialize. Further, the LRLCN-ND fails in this manner, despite its architecture being similar to an SMOE when there is one output channel (a location-dependent, softmax-weighted combination of three basis kernels). We believe the LRLCN-ND exhibits a similar gradient “mismatch” as discussed earlier (§2.2).

We now discuss a number of different ablations of the SMOE architecture and design.

**What if the “right” expert configuration is not known?** While in the above experiments, we were able to select the SMOE expert configuration (number of experts, number of selected experts, expert filter size) so that it is both necessary and sufficient to learn the task, in many situations this information may not be available. We considered alternative SMOE configurations varying each of these parameters: ❶ using six experts; ❷ experts with 5×5 kernels; ❸ and selecting two experts per location from six total. For case ❸, we summed the two SMOE output channels together.

In all three cases, the SMOE achieved 100.0 $\pm$ 0% within 1% on the heat diffusion task. In ❶, we found that they learned duplicate diffusion stencils and still routed them appropriately. ❷ learned the five-point stencil plus a boundary of near-zero values, thus being nearly identical to the 3×3 kernel. Finally, ❸ learned diffusion stencils that summed together to produce the correct diffusivity. Thus, we can see that SMOEs are robust and adapt well to these sorts of architecture choices.

**RC loss and expert error damping.** Table 1 shows results for training SMOEs with and without our routing classification loss (§2.2) and expert error damping (§2.3). Without the RC loss, SMOE performance is at par with other baselines in Fig. 3, but once it is added, performance improves significantly as the gating function now learns the location-dependency in the data. Adding expert error damping further improves performance and convergence by limiting the impact of gate misroutings

Table 3: WeatherBench [78] results (latitude-weighted RMSE).

| Model                  | Z500 [ $\text{m}^2 \text{s}^{-2}$ ] |                                 | T850 [K]                          |                                   |
|------------------------|-------------------------------------|---------------------------------|-----------------------------------|-----------------------------------|
|                        | 3 days                              | 5 days                          | 3 days                            | 5 days                            |
| Rasp and Thurey [80]   | $316 \pm 2.4$                       | $563 \pm 3.1$                   | $1.80 \pm 0.02$                   | $2.84 \pm 0.03$                   |
| → 2× wide              | $310 \pm 2.0$                       | $555 \pm 2.8$                   | $1.76 \pm 0.03$                   | $2.78 \pm 0.01$                   |
| LRLCN [30]             | $290 \pm 1.4$                       | $549 \pm 1.9$                   | $1.73 \pm 0.03$                   | $2.79 \pm 0.01$                   |
| ViT (2×2) [26]         | $438 \pm 2.8$                       | $638 \pm 3.1$                   | $2.24 \pm 0.04$                   | $2.88 \pm 0.03$                   |
| SMoE after first layer | $305 \pm 1.9$                       | $556 \pm 2.2$                   | $1.77 \pm 0.01$                   | $2.80 \pm 0.03$                   |
| Last layer SMoE        | $298 \pm 2.6$                       | $553 \pm 3.2$                   | $1.73 \pm 0.02$                   | $2.78 \pm 0.04$                   |
| 3×3 convs→SMoE         | $278 \pm 2.0$                       | $530 \pm 1.8$                   | $1.69 \pm 0.01$                   | $2.65 \pm 0.01$                   |
| → + gate prior         | <b><math>270 \pm 1.9</math></b>     | <b><math>525 \pm 2.0</math></b> | <b><math>1.66 \pm 0.02</math></b> | <b><math>2.60 \pm 0.01</math></b> |
| → rand fixed gate init | $328 \pm 3.7$                       | $572 \pm 4.1$                   | $1.89 \pm 0.08$                   | $2.96 \pm 0.05$                   |
| R&T [80] (pretrained)  | $267 \pm 1.8$                       | $500 \pm 2.4$                   | $1.66 \pm 0.03$                   | $2.43 \pm 0.02$                   |
| SMoE (pretrained)      | $253 \pm 2.1$                       | $488 \pm 1.7$                   | $1.57 \pm 0.02$                   | $2.34 \pm 0.02$                   |
| → + extra ERA5         | $232 \pm 1.5$                       | $440 \pm 1.2$                   | $1.46 \pm 0.02$                   | $2.19 \pm 0.01$                   |
| → + 1.4°               | <b><math>198 \pm 1.8</math></b>     | <b><math>382 \pm 2.0</math></b> | <b><math>1.42 \pm 0.00</math></b> | <b><math>2.06 \pm 0.02</math></b> |

Table 4: ImageNet [24] validation accuracy.

| Model                  | Top-1 %                            | Top-5 %                            |
|------------------------|------------------------------------|------------------------------------|
| ResNet-50 [42, 94]     | $80.83 \pm 0.04$                   | $95.39 \pm 0.03$                   |
| LRLCN [30]             | $80.90 \pm 0.02$                   | $95.41 \pm 0.05$                   |
| SMoE after first layer | $80.85 \pm 0.05$                   | $95.40 \pm 0.01$                   |
| Last layer SMoE        | $80.91 \pm 0.04$                   | $95.42 \pm 0.03$                   |
| 3×3 convs→SMoE         | $81.33 \pm 0.03$                   | $95.52 \pm 0.01$                   |
| Wide ResNet-50-2 [103] | <b><math>81.76 \pm 0.03</math></b> | <b><math>95.74 \pm 0.02</math></b> |

on expert learning. However, damping on its own offers little benefit, as it does not improve gate learning. These results show that these refinements are critical for good performance.

**Gating function.** Table 2 shows the performance of different gating functions (§2.1) on the SMoE. We consider six options: A single fully-connected layer (as is commonly used in MoEs [82, 86]); a single 3×3 convolution, CoordConv [65], or LCN layer; a gate with three CoordConv layers with batchnorm and ReLU; and our tensor routing gate. When training, we also considered auxiliary losses and other methods for improving performance (see §C) and report the best result. Our tensor routing offers the best performance. An LCN performs second-best, likely because it also uses separate parameters per location, but uses 9× as many parameters and requires significant computation. Other methods do not appear able to effectively capture location-dependence.

**Other ablations.** We conduct a number of additional ablation studies in §C, including using auxiliary losses and routing noise during training, routing normalizations, and expert functions.

### 3.2 Medium-Range Weather Prediction

We now discuss results on the WeatherBench [78] medium-range weather forecasting benchmark. This benchmark uses the ERA5 reanalysis dataset [44], with hourly global weather data for 1979–2018. We use the data subset suggested by Rasp et al. [78] at 5.625° resolution (32×64 grid points) and train on data from 1979–2015, validate on 2016, and report test results for 2017–2018. We otherwise follow the training methodology of Rasp and Thurey [80]. The target quantities to predict are geopotential at 500 hPa (Z500) and temperature at 850 hPa (T850) with a three- and five-day lead time. We report results using latitude-weighted root-mean-square error (RMSE).

As a baseline, we use the ResNet architecture [42] introduced by Rasp and Thurey [80], which currently reports the best results on WeatherBench. This architecture consists of 19 residual blocks each with two [3×3 convolution → LeakyReLU → batchnorm → dropout] layers, plus an initial 7×7 convolution layer. All convolutions but the last have 128 filters. We consider three additional baselines. The first is identical to the above, but with twice as many filters (256) in each convolution. Second, we replace 3×3 convolutions with LRLCN [30] layers. Finally, we use a four-layer ViT [26] with patch size 2×2, hidden dimension 1024, and eight heads (the best performing configuration).

We adapt the Rasp and Thurey ResNet to use SMoEs with three configurations: adding an SMoE layer after the first convolution; adding an SMoE layer after the final convolution; and replacing all 3×3 convolutions with SMoE layers. Each SMoE selects the same number of experts as the original layer had filters, and has twice as many experts (i.e.,  $|\mathcal{E}| = 256$ ,  $E = 128$ ). We also share the tensor routing gate across all SMoE layers with the same number of experts, so its overhead is minimal.

Because the weather data is on a fixed grid with underlying location-dependence (the Earth), we expect SMoEs to convey some benefit by specializing to the characteristics of different regions. In Table 3, we observe that this is indeed the case. Adding SMoEs improves results in all situations, with the most significant improvement coming through replacing all 3×3 convolutions with SMoEs. This showcases the advantage of incorporating appropriate location-dependent biases. Wider ResNets



Table 5: Results for prediction correction on the ENS-10 [8] dataset for ensemble post-processing.

| Metric | Model | Z500 [ $\text{m}^2 \text{s}^{-2}$ ] |                         | T850 [K]                |                         | T2m [K]                 |                         |
|--------|-------|-------------------------------------|-------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
|        |       | 5-ENS                               | 10-ENS                  | 5-ENS                   | 10-ENS                  | 5-ENS                   | 10-ENS                  |
| CRPS   | EMOS  | 79.12 $\pm$ 0.12                    | 78.80 $\pm$ 0.21        | 0.721 $\pm$ 0.01        | 0.706 $\pm$ 0.04        | 0.720 $\pm$ 0.00        | 0.711 $\pm$ 0.03        |
|        | U-Net | 76.54 $\pm$ 0.20                    | 76.18 $\pm$ 0.12        | 0.685 $\pm$ 0.00        | 0.670 $\pm$ 0.01        | 0.657 $\pm$ 0.01        | 0.644 $\pm$ 0.01        |
|        | SMoE  | <b>68.94</b> $\pm$ 0.14             | <b>67.43</b> $\pm$ 0.12 | <b>0.612</b> $\pm$ 0.01 | <b>0.590</b> $\pm$ 0.02 | <b>0.601</b> $\pm$ 0.02 | <b>0.594</b> $\pm$ 0.02 |
| EECRPS | EMOS  | 29.21 $\pm$ 0.18                    | 29.02 $\pm$ 0.13        | 0.247 $\pm$ 0.00        | 0.245 $\pm$ 0.02        | 0.244 $\pm$ 0.00        | 0.241 $\pm$ 0.02        |
|        | U-Net | 27.78 $\pm$ 0.11                    | 27.55 $\pm$ 0.19        | 0.230 $\pm$ 0.01        | 0.229 $\pm$ 0.01        | 0.225 $\pm$ 0.00        | 0.220 $\pm$ 0.01        |
|        | SMoE  | <b>23.79</b> $\pm$ 0.20             | <b>23.10</b> $\pm$ 0.16 | <b>0.207</b> $\pm$ 0.03 | <b>0.197</b> $\pm$ 0.03 | <b>0.199</b> $\pm$ 0.01 | <b>0.190</b> $\pm$ 0.02 |

offer limited improvement (in line with results reported by Rasp and Thuerey [80]). The location-dependent filters of LRLCNs improve over ResNets, but fail to match SMOEs. We were unable to achieve good performance with ViTs, but did observe that they are highly sensitive to patch size.

**Incorporating prior knowledge into gates.** While the exact nature of the location-dependence of this data is unknown, we do have a broad prior on some aspects of it, such as whether a point is land or sea. This information can be incorporated into an SMOE by initializing the tensor routing gate to bias routing to different experts. To this end, we use the land-sea mask from ERA5 to initialize the gate to route land locations to half the experts and sea locations to the other half. Note this does not fix the routing, as the gate is able to adjust as it learns. Further, the land-sea mask is already included in the input data, so all models already had access to this information.

Results with this are in the “+ gate prior” line of Table 3, and perform best. This configuration sets a new state-of-the-art for WeatherBench when not using additional data. Indeed, it nearly matches the performance of a ResNet with 150 years of additional pretraining data from climate simulations [80].

We also tried a configuration where the gate was initialized randomly and fixed rather than learned (“rand fixed gate init”). This performs worse than our baseline, as the network cannot adapt its routing choices, and each expert sees even fewer points in each sample than a standard network, resulting in less learning. Thus, learning the routing function is critically important to good performance.

**Additional data.** Following Rasp and Thuerey [80], we use 150 years of data from the MPI-ESSM-HR climate model from the CMIP6 archive [32] to pretrain our best SMOE configuration, which was then fine-tuned on ERA5 data as above. This significantly outperforms both our SMOEs without pretraining and Rasp and Thuerey’s pretrained ResNet. We incorporated more data to further push the performance by adding ERA5 data from the most recent back extension (1959–1979), increasing the dataset size by about 50%. This shows improved results; however, we suspect performance is saturating due to the coarse spatial resolution of the data. We therefore trained a final configuration with higher resolution (1.4°) data. Using this, our SMOEs *significantly outperform the state-of-the-art on WeatherBench*; indeed, its performance on T850 is very close to that of the operational Integrated Forecast System [78]. Our results are also competitive with those of Keisler [53], although they are not directly comparable (due to, e.g., different data resolutions).

**Mode collapse.** Many MoEs suffer from expert or mode collapse (e.g., [10, 82, 86]), where only a small number of experts are selected. This is typically avoided with routing noise and/or auxiliary “load balance” losses. On the heat diffusion dataset, we found these losses to offer no benefit (§C). We also did not observe mode collapse in SMOEs on WeatherBench. With the RC loss, we directly train the gate, updating routing weights toward other experts after mistakes, and so avoid such issues.

**Expert selection order.** During training, the order experts are concatenated may change (due to changes in relative routing scores, or selecting different experts), which will impact the order of channels seen by subsequent layers. When training on WeatherBench, we found this not to have a significant impact: expert order stabilizes early, allowing layers to operate on stable representations. Further, most “swapping” occurs among low-confidence experts, so is limited to a subset of channels.

### 3.3 Post-Processing Ensemble Weather Forecasts

Numerical weather prediction systems typically utilize ensembles of simulations in order to quantify uncertainty and improve forecast quality [14]. However, such ensembles typically exhibit systematic biases [92], and correcting them improves forecast skill [14, 85, 101], a task for which deep learning has shown promise [38, 79]. We use the ENS-10 dataset [8], which consists of twenty years (1998–

2017) of global reforecast [40] data at  $0.5^\circ$  spatial resolution. We follow the benchmarking setup of Ashkboos et al. [8], and correct predictions for Z500, T850, and 2 meter temperature (T2m) at a 48 hour lead-time using both five and ten ensemble members. We report results using continuous ranked probability score (CRPS) and extreme event weighted CRPS (EECRPS).

We adapt the U-Net model from the ENS-10 baselines, as it delivers good performance and operates on global data (other methods use patches). Similar to our approach for WeatherBench, **we replace each  $3\times 3$  convolution with an SMOE with four times as many experts as the original layer, and select the same number of experts as the original layer had filters**. We share tensor routing gates between all layers with the same spatial dimensions and number of experts, with the exception that the encoder and decoder trunks also use separate gates. As baselines, we use the original U-Net architecture and Ensemble Model Output Statistics (EMOS) [36], a standard post-processing method.

We observe in Table 5 that, similar to WeatherBench, SMOEs offer significant improvements in forecast skill across all situations, and set a new state-of-the-art for prediction correction on the ENS-10 dataset. This also demonstrates that SMOEs are able to scale to the very large spatial domain used by the ENS-10 data and still learn the appropriate location dependence.

### 3.4 Image Classification

Lastly, we present results on several image classification tasks; we focus here on ImageNet-1k [24] and discuss results on additional datasets in §D. While these datasets do not have a strict location-dependent structure, relaxing the strict translation equivariance of convolutions can bring benefits, and enables a direct comparison with Elsayed et al. [30]. We follow their experimental methodology and train using the recipe of Vryniotis [94]. We either insert an SMOE layer after the first or last convolutional layer of ResNet-50 [42] or replace all  $3\times 3$  convolutions with SMOE layers. Our SMOEs have twice as many experts as the original convolution layer and select half of them, to keep output dimensions constant. Gating layers are shared among all equally-sized blocks. For comparison, we also train ResNet-50 with all  $3\times 3$  convolutions replaced by LRLCN [30] layers; and a Wide ResNet-50-2 [103], which has comparable parameters to SMOEs.

Table 4 shows that SMOEs outperform LRLCNs when we replace all  $3\times 3$  convolutions, while using 56% of the parameters. However, a wide ResNet performs best overall. Nevertheless, this shows that ImageNet classification does indeed benefit from relaxing translation equivariance.

## 4 Discussion

We presented the Spatial Mixture-of-Experts layer, a novel layer that learns underlying location dependencies in data and then uses fine-grained routing to specialize experts to different areas. We also introduce a routing classification loss and expert error damping, which enable SMOEs to perform well on dense regression tasks. Prior work shows limited effectiveness on these tasks: Either it does not capture location-dependence (e.g., convolutions) or it operates at a coarse-grained level (e.g., standard MoEs). By overcoming these challenges, we show a new capability for neural networks, and set new state-of-the-arts for medium-range weather prediction and ensemble post-processing.

Many other problems of broad societal import have a similar spatial structure, particularly in scientific domains [9], and we expect SMOEs to be applicable to them. However, tasks such as facial recognition and surveillance have also historically shown benefit from such improvements [91] and SMOEs should therefore be used with care.

SMOEs show that learning location-dependence is a powerful inductive bias for certain types of data, and there are many avenues for further study. Two key areas of particular interest are to develop improved implementations for fine-grained, sparse routing; and to generalize SMOEs from operating on grids to general graphs, which would enable them to be applied to many additional tasks.

### Acknowledgements and Disclosure of Funding

We thank the members of SPCL at ETH Zürich , and Peter Dueben and Mat Chantry of ECMWF, for helpful discussions; and the anonymous reviewers for their suggestions and feedback. This work has received funding from the European High-Performance Computing Joint Undertaking (JU) under grant agreement No. 955513 (MAELSTROM), and from Huawei. N.D. received support from the ETH Postdoctoral Fellowship. We thank the Swiss National Supercomputing Center (CSCS) and Livermore Computing for computing infrastructure.

## References

- [1] Alhabib Abbas and Yiannis Andreopoulos. 2020. Biased mixtures of experts: Enabling computer vision inference under data transfer limitations. *IEEE Transactions on Image Processing* 29 (2020). arXiv:2008.09662 [cs.LG]
- [2] Shreya Agrawal, Luke Barrington, Carla Bromberg, John Burge, Cenk Gazen, and Jason Hickey. 2019. Machine learning for precipitation nowcasting from radar images. (2019). arXiv:1912.12132 [cs.CV]
- [3] Subutai Ahmad and Luiz Scheinkman. 2019. How can we be so dense? The benefits of using highly sparse representations. (2019). arXiv:1903.11257 [cs.LG]
- [4] Karim Ahmed, Mohammad Haris Baig, and Lorenzo Torresani. 2016. Network of experts for large-scale image categorization. In *European Conference on Computer Vision (ECCV)*. arXiv:1604.06119 [cs.CV]
- [5] Amjad Almahairi, Nicolas Ballas, Tim Cooijmans, Yin Zheng, Hugo Larochelle, and Aaron Courville. 2016. Dynamic capacity networks. In *International Conference on Machine Learning (ICML)*. arXiv:1511.07838 [cs.LG]
- [6] American Meteorological Society. 2022. Medium-range forecast. [https://glossary.ametsoc.org/wiki/Medium-range\\_forecast](https://glossary.ametsoc.org/wiki/Medium-range_forecast)
- [7] Mikel Artetxe, Shruti Bhosale, Naman Goyal, Todor Mihaylov, Myle Ott, Sam Shleifer, Xi Victoria Lin, Jingfei Du, Srinivasan Iyer, Ramakanth Pasunuru, et al. 2021. Efficient Large Scale Language Modeling with Mixtures of Experts. arXiv:2112.10684 [cs.CL]
- [8] Saleh Ashkboos, Langwen Huang, Nikoli Dryden, Tal Ben-Nun, Peter Dueben, Lukas Gianinazzi, Luca Kummer, and Torsten Hoeffler. 2022. ENS-10: A Dataset For Post-Processing Ensemble Weather Forecast. *Proceedings of the Neural Information Processing Systems (NeurIPS) Track on Datasets and Benchmarks*. arXiv:2206.14786 [cs.LG]
- [9] Nathan Baker, Frank Alexander, Timo Bremer, Aric Hagberg, Yannis Kevrekidis, Habib Najm, Manish Parashar, Abani Patra, James Sethian, Stefan Wild, et al. 2019. *Workshop report on basic research needs for scientific machine learning: Core technologies for artificial intelligence*. Technical Report. United States Department of Energy, Office of Science.
- [10] Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. 2016. Conditional computation in neural networks for faster models. In *International Conference on Learning Representations Workshops (ICLR-W)*. arXiv:1511.06297 [cs.LG]
- [11] Yoshua Bengio. 2013. Deep learning of representations: Looking forward. In *International conference on statistical language and speech processing*. arXiv:1305.0445 [cs.LG]
- [12] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. (2013). arXiv:1308.3432 [cs.LG]
- [13] Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. 2021. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. (2021). arXiv:2104.13478 [cs.LG]
- [14] Roberto Buizza and David Richardson. 2017. 25 years of ensemble forecasting at ECMWF. *ECMWF Newsletter* (2017). Issue 153. <https://www.ecmwf.int/node/18198>
- [15] Ke Chen, Lei Xu, and Huisheng Chi. 1999. Improved learning algorithms for mixture of experts in multiclass classification. *Neural networks* 12, 9 (1999).
- [16] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy, and Alan L Yuille. 2017. DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *IEEE transactions on pattern analysis and machine intelligence* 40, 4 (2017). arXiv:1606.00915 [cs.CV]
- [17] Yunpeng Chen, Marcus Rohrbach, Zhicheng Yan, Yan Shuicheng, Jiashi Feng, and Yannis Kalantidis. 2019. Graph-based global reasoning networks. In *Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:1811.12814 [cs.CV]
- [18] Yu-hsin Chen, Ignacio Lopez Moreno, Tara Sainath, Mirkó Visontai, Raziel Alvarez, and Carolina Parada. 2015. Locally-connected and convolutional neural networks for small footprint speaker recognition. In *INTERSPEECH*.

- [19] Zhourong Chen, Yang Li, Samy Bengio, and Si Si. 2019. You look twice: GaterNet for dynamic filter selection in CNNs. In *Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:1811.11205 [cs.CV]
- [20] Kyunghyun Cho and Yoshua Bengio. 2014. Exponentially increasing the capacity-to-computation ratio for conditional computation in deep learning. (2014). arXiv:1406.7362 [stat.ML]
- [21] Aidan Clark, Diego de las Casas, Aurelia Guy, Arthur Mensch, Michela Paganini, Jordan Hoffmann, Bogdan Damoc, Blake Hechtman, Trevor Cai, Sebastian Borgeaud, et al. 2022. Unified Scaling Laws for Routed Language Models. arXiv:2202.01169 [cs.CL]
- [22] Shail Dave, Riyadh Baghdadi, Tony Nowatzki, Sasikanth Avancha, Aviral Shrivastava, and Baoxin Li. 2021. Hardware acceleration of sparse and irregular tensor computations of ML models: A survey and insights. *Proc. IEEE* 109, 10 (2021). arXiv:2007.00864 [cs.AR]
- [23] Andrew Davis and Itamar Arel. 2013. Low-rank approximations for conditional feedforward computation in deep neural networks. In *International Conference on Learning Representations Workshops (ICLR-W)*. arXiv:1312.4461 [cs.LG]
- [24] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *Conference on computer vision and pattern recognition (CVPR)*.
- [25] Ludovic Denoyer and Patrick Gallinari. 2014. Deep sequential neural network. (2014). arXiv:1410.0510 [cs.LG]
- [26] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2021. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*. arXiv:2010.11929 [cs.CV]
- [27] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, et al. 2021. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. arXiv:2112.06905 [cs.CL]
- [28] Stéphane d’Ascoli, Hugo Touvron, Matthew L Leavitt, Ari S Morcos, Giulio Biroli, and Levent Sagun. 2021. ConViT: Improving vision transformers with soft convolutional inductive biases. In *International Conference on Machine Learning (ICML)*. arXiv:2103.10697 [cs.CV]
- [29] David Eigen, Marc’Aurelio Ranzato, and Ilya Sutskever. 2013. Learning factored representations in a deep mixture of experts. (2013). arXiv:1312.4314 [cs.LG]
- [30] Gamaleldin Elsayed, Prajit Ramachandran, Jonathon Shlens, and Simon Kornblith. 2020. Revisiting spatial invariance with low-rank local connectivity. In *International Conference on Machine Learning (ICML)*. arXiv:2002.02959 [cs.CV]
- [31] Lasse Espeholt, Shreya Agrawal, Casper Sønderby, Manoj Kumar, Jonathan Heek, Carla Bromberg, Cenk Gizen, Jason Hickey, Aaron Bell, and Nal Kalchbrenner. 2021. Skillful Twelve Hour Precipitation Forecasts using Large Context Neural Networks. (2021). arXiv:2111.07470 [cs.LG]
- [32] Veronika Eyring, Sandrine Bony, Gerald A Meehl, Catherine A Senior, Bjorn Stevens, Ronald J Stouffer, and Karl E Taylor. 2016. Overview of the Coupled Model Intercomparison Project Phase 6 (CMIP6) experimental design and organization. *Geoscientific Model Development* 9, 5 (2016).
- [33] William Fedus, Barret Zoph, and Noam Shazeer. 2021. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. arXiv:2101.03961 [cs.LG]
- [34] Chrisantha Fernando, Dylan Banarse, Charles Blundell, Yori Zwols, David Ha, Andrei A Rusu, Alexander Pritzel, and Daan Wierstra. 2017. PathNet: Evolution channels gradient descent in super neural networks. *arXiv preprint arXiv:1701.08734* (2017). arXiv:1701.08734 [cs.NE]
- [35] Hiroshi Fukui, Tsubasa Hirakawa, Takayoshi Yamashita, and Hironobu Fujiyoshi. 2019. Attention branch network: Learning of attention mechanism for visual explanation. In *Conference on computer vision and pattern recognition (CVPR)*. arXiv:1812.10025 [cs.CV]
- [36] Tilmann Gneiting, Adrian E Raftery, Anton H Westveld III, and Tom Goldman. 2005. Calibrated probabilistic forecasting using ensemble model output statistics and minimum CRPS estimation. *Monthly Weather Review* 133, 5 (2005).

- [37] Karo Gregor and Yann LeCun. 2010. Emergence of complex-like cells in a temporal product network with local receptive fields. *arXiv preprint arXiv:1006.0448* (2010). arXiv:1006.0448 [cs.NE]
- [38] Peter Grönquist, Chengyuan Yao, Tal Ben-Nun, Nikoli Dryden, Peter Dueben, Shigang Li, and Torsten Hoefler. 2021. Deep learning for post-processing ensemble weather forecasts. *Philosophical Transactions of the Royal Society A* 379, 2194 (2021). arXiv:2005.08748 [cs.LG]
- [39] Sam Gross, Marc’Aurelio Ranzato, and Arthur Szlam. 2017. Hard mixtures of experts for large scale weakly supervised vision. In *Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:1704.06363 [cs.CV]
- [40] Thomas M. Hamill, Jeffrey S. Whitaker, and Steven L. Mullen. 2006. Reforecasts: An Important Dataset for Improving Weather Predictions. *Bulletin of the American Meteorological Society* 87, 1 (2006).
- [41] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *International conference on computer vision (ICCV)*. arXiv:1502.01852 [cs.CV]
- [42] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Conference on computer vision and pattern recognition (CVPR)*. arXiv:1512.03385 [cs.CV]
- [43] Xuming He, Richard S Zemel, and Miguel A Carreira-Perpinán. 2004. Multiscale conditional random fields for image labeling. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [44] Hans Hersbach, Bill Bell, Paul Berrisford, Shoji Hirahara, András Horányi, Joaquín Muñoz-Sabater, Julien Nicolas, Carole Peubey, Raluca Radu, Dinand Schepers, et al. 2020. The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society* 146, 730 (2020).
- [45] Torsten Hoefler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. 2021. Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research* 22, 241 (2021). arXiv:2102.00554 [cs.LG]
- [46] Gary B Huang, Honglak Lee, and Erik Learned-Miller. 2012. Learning hierarchical representations for face verification with convolutional deep belief networks. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [47] Kevin Lee Hunter, Lawrence Spracklen, and Subutai Ahmad. 2021. Two Sparsities Are Better Than One: Unlocking the Performance Benefits of Sparse-Sparse Networks. (2021). arXiv:2112.13896 [cs.LG]
- [48] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. 2021. Data movement is all you need: A case study on optimizing transformers. In *Conference on Machine Learning and Systems (MLSys)*. arXiv:2007.00072 [cs.LG]
- [49] Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. 1991. Adaptive mixtures of local experts. *Neural computation* 3, 1 (1991).
- [50] Saumya Jetley, Nicholas A Lord, Namhoon Lee, and Philip HS Torr. 2018. Learn to pay attention. In *International Conference on Learning Representations (ICLR)*. arXiv:1804.02391 [cs.CV]
- [51] Xu Jia, Bert De Brabandere, Tinne Tuytelaars, and Luc V Gool. 2016. Dynamic filter networks. *Advances in neural information processing systems (NeurIPS)* (2016). arXiv:1605.09673 [cs.LG]
- [52] Michael I Jordan and Robert A Jacobs. 1994. Hierarchical mixtures of experts and the EM algorithm. *Neural computation* 6, 2 (1994).
- [53] Ryan Keisler. 2022. Forecasting Global Weather with Graph Neural Networks. (2022). arXiv:2202.07575 [physics.ao-ph]
- [54] Diederik P Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980 [cs.LG]
- [55] Alex Krizhevsky. 2009. Learning multiple layers of features from tiny images. (2009).
- [56] Mark Kurtz, Justin Kopinsky, Rati Gelashvili, Alexander Matveev, John Carr, Michael Goin, William Leiserson, Sage Moore, Nir Shavit, and Dan Alistarh. 2020. Inducing and exploiting activation sparsity for fast inference on deep neural networks. In *International Conference on Machine Learning (ICML)*.
- [57] Jeffrey K Lazo, Rebecca E Morss, and Julie L Demuth. 2009. 300 billion served: Sources, perceptions, uses, and values of weather forecasts. *Bulletin of the American Meteorological Society* 90, 6 (2009).



- [58] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998).
- [59] Dmitry Lepikhin, Hyoungho Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*. arXiv:2006.16668 [cs.CL]
- [60] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. 2021. BASE layers: Simplifying training of large, sparse models. In *International Conference on Machine Learning (ICML)*. arXiv:2103.16716 [cs.CL]
- [61] Chuan Li and Michael Wand. 2016. Combining Markov random fields and convolutional neural networks for image synthesis. In *Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:1601.04589 [cs.CV]
- [62] Jingyun Liang, Jie Zhang Cao, Guolei Sun, Kai Zhang, Luc Van Gool, and Radu Timofte. 2021. SwinIR: Image restoration using swin transformer. In *International Conference on Computer Vision (ICCV)*. arXiv:2108.10257 [eess.IV]
- [63] Tao Lin, Sebastian U Stich, Luis Barba, Daniil Dmitriev, and Martin Jaggi. 2020. Dynamic model pruning with feedback. In *International Conference on Learning Representations (ICLR)*. arXiv:2006.07253 [cs.LG]
- [64] Drew Linsley, Dan Shiebler, Sven Eberhardt, and Thomas Serre. 2019. Learning what and where to attend. In *International Conference on Learning Representations (ICLR)*. arXiv:1805.08819 [cs.CV]
- [65] Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. 2018. An intriguing failing of convolutional neural networks and the CoordConv solution. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1807.03247 [cs.CV]
- [66] Ziwei Liu, Xiao Xiao Li, Ping Luo, Chen Change Loy, and Xiaoou Tang. 2017. Deep learning Markov random field for semantic segmentation. *IEEE transactions on pattern analysis and machine intelligence* 40, 8 (2017). arXiv:1606.07230 [cs.CV]
- [67] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *International Conference on Computer Vision (ICCV)*. arXiv:2103.14030 [cs.CV]
- [68] E Majani, Ruth Erlanson, and Yaser Abu-Mostafa. 1988. On the K-winners-take-all network. In *Advances in neural information processing systems (NeurIPS)*.
- [69] Alireza Makhzani and Brendan J Frey. 2015. Winner-take-all autoencoders. In *Advances in neural information processing systems (NeurIPS)*. arXiv:1409.2752 [cs.LG]
- [70] Mason McGill and Pietro Perona. 2017. Deciding how to decide: Dynamic routing in artificial neural networks. In *International Conference on Machine Learning (ICML)*. arXiv:1703.06217 [stat.ML]
- [71] Jiquan Ngiam, Zhenghao Chen, Daniel Chia, Pang Koh, Quoc Le, and Andrew Ng. 2010. Tiled convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)* (2010).
- [72] Roman Novak, Lechao Xiao, Jaehoon Lee, Yasaman Bahri, Greg Yang, Jiri Hron, Daniel A Abolafia, Jeffrey Pennington, and Jascha Sohl-Dickstein. 2019. Bayesian deep convolutional networks with many channels are gaussian processes. In *International Conference on Learning Representations (ICLR)*. arXiv:1810.05148 [stat.ML]
- [73] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems (NeurIPS)*. arXiv:1912.01703 [cs.LG]
- [74] Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, et al. [n.d.]. FourCastNet: A global data-driven high-resolution weather model using adaptive fourier neural operators. ([n.d.]). arXiv:2202.11214

- [75] Svetlana Pavlitskaya, Christian Hubschneider, Michael Weber, Ruby Moritz, Fabian Huger, Peter Schlicht, and Marius Zollner. 2020. Using mixture of expert models to gain insights into semantic segmentation. In *Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*.
- [76] Prajit Ramachandran and Quoc V Le. 2018. Diversity and depth in per-example routing models. In *International Conference on Learning Representations (ICLR)*.
- [77] Marc’Aurelio Ranzato, Joshua Susskind, Volodymyr Mnih, and Geoffrey Hinton. 2011. On deep generative models with applications to recognition. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [78] Stephan Rasp, Peter D Dueben, Sebastian Scher, Jonathan A Weyn, Soukayna Mouatadid, and Nils Thuerey. 2020. WeatherBench: a benchmark data set for data-driven weather forecasting. *Journal of Advances in Modeling Earth Systems* 12, 11 (2020). arXiv:2002.00469 [physics.ao-ph]
- [79] Stephan Rasp and Sebastian Lerch. 2018. Neural networks for postprocessing ensemble weather forecasts. *Monthly Weather Review* 146, 11 (2018). arXiv:1805.09091 [stat.ML]
- [80] Stephan Rasp and Nils Thuerey. 2021. Data-driven medium-range weather prediction with a ResNet pretrained on climate simulations: A new model for WeatherBench. *Journal of Advances in Modeling Earth Systems* 13, 2 (2021). arXiv:2008.08626 [physics.ao-ph]
- [81] Suman Ravuri, Karel Lenc, Matthew Willson, Dmitry Kangin, Remi Lam, Piotr Mirowski, Megan Fitzsimons, Maria Athanassiadou, Sheleem Kashem, Sam Madge, et al. 2021. Skilful precipitation nowcasting using deep generative models of radar. *Nature* 597, 7878 (2021). arXiv:2104.00954 [cs.LG]
- [82] Carlos Riquelme, Joan Puigcerver, Basil Mustafa, Maxim Neumann, Rodolphe Jenatton, André Susano Pinto, Daniel Keysers, and Neil Houlsby. 2021. Scaling Vision with Sparse Mixture of Experts. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2106.05974 [cs.CV]
- [83] Clemens Rosenbaum, Tim Klinger, and Matthew Riemer. 2018. Routing networks: Adaptive selection of non-linear functions for multi-task learning. In *International Conference on Learning Representations (ICLR)*. arXiv:1711.01239 [cs.LG]
- [84] Christoph Schär, Oliver Fuhrer, Andrea Arteaga, Nikolina Ban, Christophe Charpillot, Salvatore Di Girolamo, Laureline Hentgen, Torsten Hoefler, Xavier Lapillonne, David Leutwyler, et al. 2020. Kilometer-scale climate models: Prospects and challenges. *Bulletin of the American Meteorological Society* 101, 5 (2020).
- [85] MG Schultz, Clara Betancourt, Bing Gong, Felix Kleinert, Michael Langguth, LH Leufen, Amirpasha Mozaffari, and Scarlet Stadler. 2021. Can deep learning beat numerical weather prediction? *Philosophical Transactions of the Royal Society A* 379, 2194 (2021).
- [86] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*. arXiv:1701.06538 [cs.LG]
- [87] Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-Kin Wong, and Wang-chun Woo. 2015. Convolutional LSTM network: A machine learning approach for precipitation nowcasting. In *Advances in neural information processing systems (NeurIPS)*. arXiv:1506.04214 [cs.CV]
- [88] Xingjian Shi, Zhihan Gao, Leonard Lausen, Hao Wang, Dit-Yan Yeung, Wai-kin Wong, and Wang-chun Woo. 2017. Deep learning for precipitation nowcasting: A benchmark and a new model. In *Advances in neural information processing systems (NeurIPS)*. arXiv:1706.03458 [cs.CV]
- [89] Casper Kaae Sønderby, Lasse Espeholt, Jonathan Heek, Mostafa Dehghani, Avital Oliver, Tim Salimans, Shreya Agrawal, Jason Hickey, and Nal Kalchbrenner. 2020. MetNet: A neural weather model for precipitation forecasting. (2020). arXiv:2003.12140 [cs.LG]
- [90] Suraj Srinivas, Akshayvarun Subramanya, and R Venkatesh Babu. 2017. Training sparse neural networks. In *Conference on computer vision and pattern recognition workshops (CVPRW)*. arXiv:1611.06694 [cs.CV]
- [91] Yaniv Taigman, Ming Yang, Marc’Aurelio Ranzato, and Lior Wolf. 2014. DeepFace: Closing the gap to human-level performance in face verification. In *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [92] Zoltan Toth and Eugenia Kalnay. 1993. Ensemble forecasting at NMC: The generation of perturbations. *Bulletin of the American Meteorological Society* 74, 12 (1993).

- [93] Ashish Vaswani, Prajit Ramachandran, Aravind Srinivas, Niki Parmar, Blake Hechtman, and Jonathon Shlens. 2021. Scaling local self-attention for parameter efficient visual backbones. In *Conference on Computer Vision and Pattern Recognition (CVPR)*. arXiv:2103.12731 [cs.CV]
- [94] Vasilis Vryniotis. 2021. How to Train State-of-the-Art Models Using TorchVision’s Latest Primitives. <https://pytorch.org/blog/how-to-train-state-of-the-art-models-using-torchvision-latest-primitives/>
- [95] Fei Wang, Mengqing Jiang, Chen Qian, Shuo Yang, Cheng Li, Honggang Zhang, Xiaogang Wang, and Xiaoou Tang. 2017. Residual attention network for image classification. In *Conference on computer vision and pattern recognition (CVPR)*. arXiv:1704.06904 [cs.CV]
- [96] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. 2021. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *International Conference on Computer Vision (ICCV)*. arXiv:2102.12122 [cs.CV]
- [97] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. 2022. PVT v2: Improved baselines with Pyramid Vision Transformer. *Computational Visual Media* (2022). arXiv:2106.13797 [cs.CV]
- [98] Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. 2018. Non-local neural networks. In *Conference on computer vision and pattern recognition (CVPR)*. arXiv:1711.07971 [cs.CV]
- [99] Xin Wang, Fisher Yu, Lisa Dunlap, Yi-An Ma, Ruth Wang, Azalia Mirhoseini, Trevor Darrell, and Joseph E Gonzalez. 2020. Deep mixture of experts via shallow embedding. In *Uncertainty in Artificial Intelligence*. arXiv:1806.01531 [cs.CV]
- [100] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. 2018. CBAM: Convolutional block attention module. In *European conference on computer vision (ECCV)*. arXiv:1807.06521 [cs.CV]
- [101] World Meteorological Organization. 2021. *Guidelines on Ensemble Prediction System Postprocessing, 2021 edition*. Technical Report. World Meteorological Organization.
- [102] Brandon Yang, Gabriel Bender, Quoc V Le, and Jiquan Ngiam. 2019. CondConv: Conditionally parameterized convolutions for efficient inference. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:1904.04971 [cs.CV]
- [103] Sergey Zagoruyko and Nikos Komodakis. 2016. Wide residual networks. In *British Machine Vision Conference (BMVC)*. arXiv:1605.07146 [cs.CV]
- [104] Songyang Zhang, Xuming He, and Shipeng Yan. 2019. LatentGNN: Learning efficient non-local relations for visual recognition. In *International Conference on Machine Learning (ICML)*. arXiv:1905.11634 [cs.CV]
- [105] Kaili Zhao, Wen-Sheng Chu, and Honggang Zhang. 2016. Deep region and multi-label learning for facial action unit detection. In *Conference on computer vision and pattern recognition (CVPR)*.
- [106] Shuai Zheng, Sadeep Jayasumana, Bernardino Romera-Paredes, Vibhav Vineet, Zhizhong Su, Dalong Du, Chang Huang, and Philip HS Torr. 2015. Conditional random fields as recurrent neural networks. In *International Conference on Computer Vision (ICCV)*.
- [107] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. Designing effective sparse expert models. arXiv:2202.08906 [cs.CL]

Table B.1: SMOE performance when initializing the gate or experts either randomly or perfectly.

| Gate           |         | Experts        |         | % within 1%             |
|----------------|---------|----------------|---------|-------------------------|
| Initialization | Frozen? | Initialization | Frozen? |                         |
| Random         | ✗       | Random         | ✗       | $91.5^{\pm 0.4}$        |
| Random         | ✗       | Perfect        | ✗       | $91.4^{\pm 0.4}$        |
| Random         | ✗       | Perfect        | ✓       | $80.9^{\pm 0.2}$        |
| Perfect        | ✗       | Random         | ✗       | $96.0^{\pm 0.3*}$       |
| Perfect        | ✓       | Random         | ✗       | <b>100.0</b> $^{\pm 0}$ |
| Perfect        | ✗       | Perfect        | ✗       | <b>100.0</b> $^{\pm 0}$ |

\* Unlike other configurations, after reaching this performance, the model performance degraded and converged to  $91.6^{\pm 0.3}\%$  within 1%.

## A Training Details

Here we provide additional details on the training setups for experiments in §3.

### A.1 Medium-Range Weather Prediction

We follow the training setup of Rasp and Thuerey [80]. We use a batch size of 64, the Adam [54] optimizer with an initial learning rate of 0.001, which we divide by a factor of 10 if the validation loss has not improved for two epochs to a minimum learning rate of 1e-6, and early stopping if the validation loss does not improve for five epochs.

Data consists of geopotential, temperature, the  $u$ - $v$  component of wind, specific humidity, top-of-atmosphere incident solar radiation, 2-meter temperature, 6-hourly accumulated precipitation, a land-sea mask, orography, and the latitude/longitude. Multi-level variables are given at seven vertical levels (50, 250, 500, 600, 700, 800, and 925 hPa). We use three timesteps as input context, consisting of  $(t - 12, t - 6, t - 0)$  hours. Data are concatenated together channel-wise to form network input.

### A.2 Post-Processing Ensemble Weather Forecasts

We follow the training setup of Ashkboos et al. [8]. All models were trained for ten epochs with Adam [54] with a learning rate of  $10^{-5}$  and a batch size of 8 for the U-Net and SMOE, and batch size 1 for EMOS.

The ENS-10 data consists of temperature, geopotential, specific humidity, vertical velocity, divergence, and the  $u$  and  $v$  wind components at eleven vertical levels (10, 50, 100, 200, 300, 400, 500, 700, 850, 925, and 1000 hPa); and the sea surface temperature, total column water, total column water vapor, convective precipitation, mean sea level pressure, total cloud cover, 10 m  $u$  and  $v$  wind components, 2 m temperature, total precipitation, and skin temperature at surface on a single surface level. The ground truth data used is ERA5 data specified similarly to the ENS-10 reforecast data.

### A.3 Image Classification

Our ResNet training recipe is adapted directly from that of Vryniotis [94]. We use a batch size of 128 per GPU (and a total batch size of 8192) and trained for 600 epochs using the SGD optimizer with a learning rate of 0.5 (and a linear warmup over five epochs) and momentum (factor 0.9), a cosine annealing learning rate schedule, weight decay factor 2e-5, and elastic model averaging.

## B Routing Challenges

We have empirically observed that, without using the routing classification loss (§2.2) and instead training a gate and experts with the same loss, good gating functions are challenging to learn, particularly for regression tasks. We hypothesize that this is due to a gradient “mismatch”, where gradients are informative for experts but not the gate. Here we first present a simple ablation that gives some evidence that this is the case, then discuss a possible explanation.

In Table B.1, we show results for training an SMOE on the heat diffusion task, with a setup identical to that of §3.1, but *without* the RC loss or expert error damping, when initializing the gate and/or experts to be either random or to a perfect, correct initialization. (Note that such an initialization is

Table C.1: Effect of different SMOE routing normalizations.

| Routing Normalization | % within 1%          |
|-----------------------|----------------------|
| Softmax               | <b>100.0</b> $\pm 0$ |
| Abs. value            | 90.2 $\pm 0.3$       |
| Softmax + Abs. value  | <b>100.0</b> $\pm 0$ |
| None                  | <b>100.0</b> $\pm 0$ |

Table C.2: Effect of aux. losses and noise on SMOEs with the RC loss and expert error damping.

| Importance | Load* | Spatial Agreement | Noise | % within 1%          |
|------------|-------|-------------------|-------|----------------------|
| X          | X     | X                 | X     | <b>100.0</b> $\pm 0$ |
| ✓          | X     | X                 | X     | <b>100.0</b> $\pm 0$ |
| ✓          | X     | X                 | ✓     | 95.7 $\pm 0.2$       |
| ✓          | ✓     | X                 | ✓     | 95.7 $\pm 0.3$       |
| X          | ✓     | X                 | ✓     | 95.7 $\pm 0.2$       |
| X          | X     | X                 | ✓     | 95.7 $\pm 0.4$       |
| ✓          | X     | ✓                 | X     | <b>100.0</b> $\pm 0$ |
| ✓          | X     | ✓                 | ✓     | 95.6 $\pm 0.5$       |
| ✓          | ✓     | ✓                 | ✓     | 95.7 $\pm 0.3$       |
| X          | ✓     | ✓                 | ✓     | 95.7 $\pm 0.1$       |
| X          | X     | ✓                 | ✓     | 95.7 $\pm 0.2$       |

\* The load loss is only defined when there is routing noise.

Table C.3: Effect of aux. losses and noise on SMOEs without the RC loss and expert error damping.

| Importance | Load* | Spatial Agreement | Noise | % within 1%    |
|------------|-------|-------------------|-------|----------------|
| X          | X     | X                 | X     | 91.5 $\pm 0.4$ |
| ✓          | X     | X                 | X     | 90.8 $\pm 0.6$ |
| ✓          | X     | X                 | ✓     | 74.0 $\pm 0.2$ |
| ✓          | ✓     | X                 | ✓     | 73.9 $\pm 0.5$ |
| X          | ✓     | X                 | ✓     | 74.2 $\pm 0.3$ |
| X          | X     | X                 | ✓     | 73.9 $\pm 0.5$ |
| ✓          | X     | ✓                 | X     | 91.3 $\pm 0.2$ |
| ✓          | X     | ✓                 | ✓     | 74.1 $\pm 0.2$ |
| ✓          | ✓     | ✓                 | ✓     | 73.7 $\pm 0.3$ |
| X          | ✓     | ✓                 | ✓     | 74.0 $\pm 0.4$ |
| X          | X     | ✓                 | ✓     | 73.9 $\pm 0.3$ |

known because of how the heat diffusion dataset is generated.) We can see that a random initialization for both does not perform well, while a perfect initialization achieves perfect accuracy. However, a perfect initialization of the experts alone does not result in good performance. Indeed, freezing expert weights at their perfect initialization performs *worse*, as the gate is unable to learn to route each location to the correct expert. Initializing the gate perfectly results in better performance, but with continued training, the quality of the gate degrades; if the gate is instead frozen, the SMOE easily converges to perfect accuracy. We thus see that the gating function appears to be harder to learn and that when training end-to-end with a single loss, the gradients result in bad gate updates.

We hypothesize that this occurs because the gate is attempting to learn a classification task (“which expert(s) should be selected at a point?”), but the experts are learning a regression task with mean-square error. In such a setting, the sign of a gradient is not informative about whether a routing decision was correct, as there is no thresholding for the experts, but there is for the gate (due to the top- $E$  routing). Hence, training an SMOE with a single loss is likely to lead to poor performance.

## C Additional SMOE Ablation Studies

Here we present additional ablations on SMOE configurations. All results use the standard configuration in §3 except where noted. We also tried gating functions other than tensor routing, but did not see improved performance.

**Routing normalization.** We compare different approaches for normalizing the gating function (i.e., normalizing  $g(x)$ , see §2.1) in Table C.1. We use no normalization, as that is both simplest and performs the best.

**Auxiliary losses.** We compare different MoE auxiliary routing losses, plus using routing noise, in Table C.2. We define these auxiliary losses precisely in §C.1. We can see that auxiliary losses neither help nor hurt SMOE performance; however, adding noise results in significant performance degradation. In Table C.3, we show SMOE training with different auxiliary losses and routing noise when *not* using the RC loss and expert error damping. Auxiliary losses in this setting offer no improvement over baseline performance, showing they cannot replace the RC loss.

**Expert function.** We compare between using standard convolution and CoordConv [65] as the SMOE experts in Table C.4.

**Weighted SMOEs.** We compare between weighted and unweighted SMOEs in Table C.5. Both formulations perform the same, so we prefer the simpler, unweighted version; however, on other tasks, weighted SMOEs may perform better.



Table C.4: Effect of different SMOE expert functions.

| Expert function | % within 1%    |
|-----------------|----------------|
| Convolution     | 100.0 $\pm$ 0  |
| CoordConv [65]  | 96.5 $\pm$ 0.3 |

Table C.5: Weighted versus unweighted SMOEs.

| Weighted? | % within 1%   |
|-----------|---------------|
| ✓         | 100.0 $\pm$ 0 |
| ✗         | 100.0 $\pm$ 0 |

### C.1 Auxiliary Losses

Here we describe the auxiliary losses we explored. The importance and load losses follow the formulation of Riquelme et al. [82]; the spatial agreement loss is a novel auxiliary loss we explored. Note that these losses typically aim to enforce a balanced usage of experts; while this may be appropriate for other MoEs, it is *not* necessarily useful for SMOEs, which may have experts that specialize to rare location types.

#### C.1.1 Importance Loss

The importance loss encourages the gate to assign equal importance to each expert, where the importance is the (normalized) routing weight for an expert  $e$  over a batch  $X$ :

$$\text{Imp}_e(X) = \sum_{x \in X} G(x)_e.$$

Then the importance loss is the square of the coefficient of variation of the expert importances:

$$\mathcal{L}_{\text{Imp}}(X) = \left( \frac{\text{std}(\text{Imp}(X))}{\text{mean}(\text{Imp}(X))} \right)^2.$$

#### C.1.2 Load Loss

The load loss attempts to further encourage balanced usage of experts. However, as this quantity is not differentiable, we instead use the probability of an expert  $e$  being selected if only the routing noise were resampled. Let  $T$  be the threshold above which experts were selected at a point (i.e., the  $E$ th maximum routing score) during the original forward pass. Then the probability of expert  $e$  being above this threshold if we resample the original noise  $\varepsilon$  to be  $\varepsilon'$  is

$$p_e(x) = P(G(x)_e + \varepsilon' \geq T) = P(\varepsilon' \geq T - G(x)_e).$$

We then define the load over a batch  $X$  as

$$\text{Load}_e(X) = \sum_{x \in X} p_e(x)$$

and finally the load loss as the squared coefficient of variation:

$$\mathcal{L}_{\text{Load}}(X) = \left( \frac{\text{std}(\text{Load}(X))}{\text{mean}(\text{Load}(X))} \right)^2.$$

#### C.1.3 Spatial Agreement Loss

We introduce the spatial agreement loss, which is conceptually similar to the above losses, to encourage the gate to select the same experts at a given location across samples in a batch. This loss,  $\mathcal{L}_{\text{SA}}$ , is the standard deviation of the routing weights for each expert at each point, averaged across each point, and then summed over experts. We considered using the coefficient of variation rather than standard deviation for this, but found that the mean was often very close to zero, making optimization challenging.

#### C.1.4 Final Auxiliary Loss

We weight each used auxiliary loss equally, then add them to the final network loss, scaled by 0.01, as done in Riquelme et al. [82]. We did not find the final performance to be sensitive to the choice of scaling factor.

## D Additional Image Classification Results

We report image classification results for SMOEs on two additional tasks, MNIST [58] (Table D.1) and CIFAR10 [55] (Table D.2). We follow the experimental setup of Elsayed et al. [30] for these

| Table D.1: MNIST results. |                         | Table D.2: CIFAR10 results. |                         |
|---------------------------|-------------------------|-----------------------------|-------------------------|
| Model                     | Accuracy %              | Model                       | Accuracy %              |
| Convolution               | 98.86 $\pm 0.02$        | Convolution                 | 78.83 $\pm 0.07$        |
| LCN                       | 99.09 $\pm 0.02$        | LCN                         | 72.03 $\pm 0.10$        |
| CoordConv [65]            | 99.40 $\pm 0.03$        | CoordConv [65]              | 80.98 $\pm 0.09$        |
| Wide convolution          | 99.15 $\pm 0.01$        | Wide convolution            | 82.43 $\pm 0.13$        |
| LRLCN [30]                | 99.49 $\pm 0.01$        | LRLCN [30]                  | 84.85 $\pm 0.08$        |
| SMoE                      | <b>99.54</b> $\pm 0.01$ | SMoE                        | <b>85.05</b> $\pm 0.07$ |

datasets, and use a network of three convolution layers, 64 channels per layer, and  $3 \times 3$  filters, with batch normalization and ReLU activations. The final classification layer consists of global average pooling followed by a linear layer. Models used a mini-batch size of 512 and otherwise followed our standard training setup.

For our SMoE, we replaced all convolution layers with SMoEs with shared gates and 128 experts per layer (selecting 64). For the LRLCN, we replaced all layers with LRLCNs with spatial rank 4, which we found to perform best. The LCN network consisted of two convolutions followed by a third LCN layer, which performed best in Elsayed et al. [30].